Initiation à la programmation en Python

Des premières instructions aux listes

```
def bonjour(nom):
    return "Bonjour " + nom + " !"

for eleve in classe:
    print(bonjour(eleve))
```

Table des matières

1 Découvrir Python **5**
- 1.1 Introduction historique . . . 5
  - 1.1.1 De la machine qui calcule à la machine qu'on programme . . . 5
  - 1.1.2 Python, de 1991 à nos jours . . . 5
- 1.2 Qu'est-ce qu'un programme ? . . . 6
  - 1.2.1 Instructions et interpréteur . . . 6
  - 1.2.2 Deux façons de travailler . . . 6
- 1.3 Afficher avec print . . . 6
- 1.4 Commentaires et lisibilité . . . 7
- 1.5 Lire un message d'erreur . . . 7

2 Variables, types et calculs **9**
- 2.1 Variables et affectation . . . 9
  - 2.1.1 Ranger une valeur . . . 9
  - 2.1.2 Nommer ses variables . . . 10
- 2.2 Les types de données . . . 10
  - 2.2.1 Passer d'un type à l'autre . . . 11
- 2.3 Les opérations . . . 11
  - 2.3.1 Le module `math` . . . 12
- 2.4 Dialoguer avec l'utilisateur . . . 12

3 Tests et conditions **13**
- 3.1 Booléens et comparaisons . . . 13
- 3.2 Combiner des conditions . . . 14
- 3.3 L'instruction conditionnelle . . . 14
  - 3.3.1 La forme simple . . . 14
  - 3.3.2 Les formes `else` et `elif` . . . 15
- 3.4 Exemples rédigés . . . 15

4 Boucles bornées : `for` **17**
- 4.1 Parcourir une plage d'entiers . . . 17
- 4.2 La boucle `for` . . . 18
- 4.3 Accumuler un résultat . . . 18
- 4.4 Calculer les termes d'une suite . . . 19
- 4.5 Boucles imbriquées . . . 20

5 Boucles non bornées : `while` **21**
- 5.1 Répéter tant qu'une condition est vraie . . . 21
- 5.2 Rechercher un seuil . . . 22
- 5.3 `for` ou `while` ? . . . 23
- 5.4 Deux algorithmes d'arithmétique . . . 23

6 Les fonctions **25**
6.1 Définir et appeler une fonction . . . 25
6.2 `return` n'est pas `print` . . . 26
6.3 Plusieurs paramètres, plusieurs résultats . . . 26
6.4 La portée des variables . . . 27
6.5 Réutiliser des fonctions existantes . . . 27

7 Les listes **29**
7.1 Créer et lire une liste . . . 29
7.2 Modifier une liste . . . 30
7.3 Extraire une tranche . . . 30
7.4 Construire une liste avec une boucle . . . 30
7.5 Parcourir une liste . . . 31
7.6 Un piège : deux noms pour une même liste . . . 31

8 Quelques algorithmes classiques **33**
8.1 Le maximum d'une liste . . . 33
8.2 Balayage : encadrer une solution . . . 33
8.3 La dichotomie . . . 34
8.4 Le tri par sélection . . . 35
8.5 Simulation et loi des grands nombres . . . 35
8.6 L'aire sous une courbe . . . 36

A Erreurs fréquentes et mémento **37**
A.1 Les messages d'erreur les plus courants . . . 37
A.2 Mémento . . . 38
A.3 Six règles pour programmer sans souffrir . . . 38

B Deux environnements **39**
B.1 Programiz, dans le navigateur . . . 39
B.1.1 Ouvrir l'éditeur . . . 39
B.1.2 Suivre l'exécution pas à pas . . . 40
B.1.3 Les limites à connaître . . . 40
B.2 Thonny, sur son ordinateur . . . 40
B.2.1 Installer le logiciel . . . 40
B.2.2 L'interface . . . 41
B.2.3 Écrire et exécuter un premier programme . . . 41
B.2.4 Le débogueur, ou le tableau de trace automatique . . . 41
B.3 Lequel choisir ? . . . 42

Chapitre 1

Découvrir Python

1.1 Introduction historique

> *La machine analytique n'a aucune prétention à créer quoi que ce soit. Elle peut faire tout ce que nous savons lui ordonner d'accomplir.*
>
> Ada Lovelace - *Note G* des *Sketch of the Analytical Engine*, Scientific Memoirs, 1843

Un ordinateur ne comprend rien. Il exécute, à une vitesse prodigieuse, une suite d'ordres élémentaires que quelqu'un a dû écrire avant lui, dans les moindres détails, sans laisser place au sous-entendu. Programmer, c'est précisément apprendre à écrire ces ordres : c'est un exercice de rigueur qui ressemble beaucoup à la rédaction d'une démonstration, et qui a sur elle un avantage rare, celui de pouvoir être vérifié immédiatement.

1.1.1 De la machine qui calcule à la machine qu'on programme

Les machines à calculer sont anciennes : la *Pascaline* de **Blaise Pascal**, en 1642, additionne et soustrait au moyen d'engrenages. Mais elle ne fait que cela, et elle le fait toujours de la même façon. L'idée décisive vient de **Charles Babbage**, qui conçoit vers 1834 une *machine analytique* capable d'enchaîner des opérations selon un plan fourni de l'extérieur, sur des cartes perforées. La machine ne sera jamais construite, mais **Ada Lovelace**, qui traduit en 1843 un mémoire de l'ingénieur italien Menabrea, y ajoute des notes trois fois plus longues que le texte original. Dans la dernière, la fameuse *Note G*, elle décrit pas à pas la suite d'opérations qui calculerait les nombres de Bernoulli : c'est ce qu'on considère aujourd'hui comme le premier programme de l'histoire, écrit un siècle avant que n'existe une machine capable de l'exécuter.

Il faut attendre les années 1940 et les travaux d'**Alan Turing** et de **John von Neumann** pour que la machine programmable devienne réalité. Les premiers ordinateurs se programment alors dans leur propre langue, une suite de nombres appelée *langage machine*, illisible pour un humain. La naissance des **langages de programmation** — le Fortran en 1957, le C en 1972 — répond à un besoin simple : écrire des instructions que l'on puisse relire, corriger et transmettre.

1.1.2 Python, de 1991 à nos jours

Pendant les vacances de Noël 1989, le Néerlandais **Guido van Rossum**, chercheur au *Centrum Wiskunde & Informatica* d'Amsterdam, entreprend d'écrire un langage de programmation pour occuper ses journées de congé. Il le veut lisible, concis, et facile à apprendre. La première version publique paraît en 1991. Quant au nom, il ne doit rien au serpent : van Rossum était un lecteur assidu des scripts des *Monty Python*, la troupe comique britannique, et cherchait un nom court et un peu irrévérencieux.

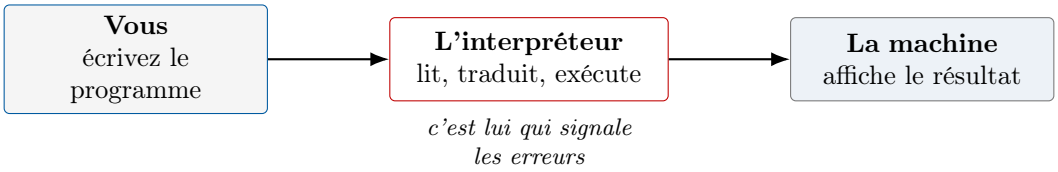
Python est aujourd'hui l'un des langages les plus employés au monde, de l'analyse de données à l'intelligence artificielle. C'est aussi, depuis 2009 en France, le langage retenu pour l'enseignement de l'algorithmique au lycée, et c'est à ce titre qu'il nous intéresse : il va nous permettre de faire effectuer à la machine les calculs répétitifs que nous ne pourrions pas mener à la main, et de tester nos conjectures avant de les démontrer.

1.2 Qu'est-ce qu'un programme ?

1.2.1 Instructions et interpréteur

> **Définition :** Une **instruction** est un ordre élémentaire que la machine sait exécuter. Un **programme** est une suite finie d'instructions, écrites dans un ordre précis. Le **langage** est la grammaire à laquelle ces instructions doivent obéir.

> **Définition :** L'**interpréteur** Python est le programme qui lit votre texte, le traduit en langage machine et l'exécute, *une instruction après l'autre, de haut en bas.*



La figure ci-dessus se lit de gauche à droite : rien ne se passe tant que l'interpréteur n'a pas été lancé, et tout ce que vous verrez à l'écran est ce que le programme aura explicitement demandé d'afficher.

> *Remarque :* L'interpréteur ne devine jamais votre intention. S'il rencontre un ordre qu'il ne comprend pas, il s'arrête et affiche un message ; s'il comprend un ordre différent de celui que vous aviez en tête, il l'exécute sans broncher. Les deux situations sont fréquentes, et la seconde est de loin la plus redoutable.

1.2.2 Deux façons de travailler

On peut s'adresser à Python de deux manières. En **mode interactif**, on tape une instruction dans une console, on appuie sur la touche Entrée, et la réponse apparaît immédiatement. L'invite de commande est traditionnellement notée `>>>`. Ce mode est idéal pour essayer quelque chose ou vérifier une valeur.

```
>>> 3 + 5
8
>>> 2 ** 10
1024
```

En **mode script**, on écrit l'ensemble du programme dans un fichier d'extension `.py`, puis on l'exécute d'un seul bloc. C'est le mode que nous emploierons dès qu'un programme dépassera deux lignes, car il permet de corriger et de relancer sans tout retaper.

> *Remarque :* En mode interactif, Python affiche spontanément la valeur de ce que vous tapez. En mode script, il n'affiche *rien* de lui-même : il faut le lui demander avec l'instruction `print`, que nous découvrons maintenant. Un programme qui « ne fait rien » a très souvent simplement oublié d'afficher son résultat.

1.3 Afficher : l'instruction `print`

> **Syntaxe :** L'instruction `print` affiche à l'écran ce qu'on lui donne entre parenthèses.
>
> ```
> print("Bonjour tout le monde")
> print(3 + 5)
> ```

Les guillemets sont essentiels : ils délimitent un **texte**, que Python recopie tel quel sans chercher à le comprendre. Sans guillemets, Python interprète ce qu'il lit comme un calcul et affiche le résultat. La différence saute aux yeux sur cet exemple.

```
print("3 + 5")
print(3 + 5)
```

Affichage obtenu :

```
3 + 5
8
```

Exemple : On peut donner à `print` plusieurs éléments séparés par des virgules : ils sont affichés à la suite, séparés par une espace. Le programme

```
print("Le double de", 7, "vaut", 2 * 7)
```

produit l'affichage `Le double de 7 vaut 14`. Remarquez que le texte est recopié, tandis que le calcul `2 * 7` est effectué avant l'affichage : Python évalue toujours ce qu'il peut évaluer avant d'afficher quoi que ce soit.

Remarque : Chaque `print` termine par un passage à la ligne : deux instructions `print` successives donnent deux lignes d'affichage. Un `print()` vide, sans rien entre les parenthèses, produit une ligne blanche, ce qui est commode pour aérer un affichage un peu long.

Remarque : L'erreur la plus répandue chez les débutants consiste à oublier une parenthèse ou un guillemet fermants. Prenez tout de suite l'habitude de les écrire par paire : tapez `print()` puis revenez placer le contenu à l'intérieur, plutôt que d'écrire de gauche à droite en espérant ne rien oublier.

1.4 Commentaires et lisibilité

> **Définition :** Un **commentaire** est un texte destiné au lecteur humain et ignoré par l'interpréteur. En Python, tout ce qui suit le caractère `#` sur une ligne est un commentaire.

```
# Programme d'accueil - version du 3 septembre
print("Bonjour")      # on salue l'utilisateur
```

Un commentaire utile ne répète pas le code : il explique *pourquoi* on fait quelque chose, ou rappelle la formule mathématique dont une ligne est la traduction. Écrire `# on ajoute 1 à n` au-dessus de la ligne `n = n + 1` n'apporte rien ; écrire `# n compte le nombre d'essais déjà effectués` éclaire tout le programme.

Remarque : Le commentaire sert aussi à neutraliser provisoirement une ligne que l'on soupçonne fautive, sans l'effacer. C'est une technique de recherche d'erreur que nous emploierons au chapitre 8.

1.5 Lire un message d'erreur

Se tromper en programmant n'est pas un accident, c'est l'état normal du travail. Ce qui distingue un programmeur expérimenté d'un débutant n'est pas l'absence d'erreurs, mais la vitesse à laquelle il les identifie. Or Python fournit pour cela un message précieux, que la plupart des élèves commencent par ne pas lire.

Supposons que le fichier `essai.py` contienne les deux lignes suivantes, la seconde faisant appel à une variable `prix` qui n'a jamais été définie.

```
print("bonjour")
print(prix)
```

L'exécution affiche d'abord `bonjour`, puis s'interrompt sur ce message :

```
Traceback (most recent call last):
  File "essai.py", line 2, in <module>
    print(prix)
          ^^^^
NameError: name 'prix' is not defined
```

♡ **Méthode :** Un message d'erreur se lit **de bas en haut**, et il comporte trois informations.

— La **dernière ligne** donne le type de l'erreur et sa cause : ici, le nom `prix` n'est pas défini.

— La **ligne indiquée** précise où Python a buté : ligne 2 du fichier `essai.py`. L'erreur peut avoir été commise plus haut, mais c'est là qu'elle se manifeste.

— Les **accents circonflexes** soulignent l'élément précis qui pose problème.

Exemple : Voici les trois erreurs que l'on rencontre le plus tôt. Une `SyntaxError` signale une phrase mal formée, par exemple un deux-points oublié après un test : Python affiche alors `SyntaxError: expected ':'`. Une `NameError` signale un nom inconnu, le plus souvent une faute de frappe sur le nom d'une variable. Une `TypeError` signale une opération impossible entre deux natures d'objets : l'instruction `print("2" + 3)` déclenche le message `TypeError: can only concatenate str (not "int") to str`, autrement dit « on ne peut ajouter du texte qu'à du texte ». Nous comprendrons précisément pourquoi au chapitre suivant.

Remarque : Une erreur de syntaxe est détectée *avant* toute exécution : si votre programme contient une telle faute à la ligne 40, rien du tout ne s'exécutera, pas même la ligne 1. Une `NameError`, au contraire, survient pendant l'exécution : les lignes précédentes ont bien été exécutées, comme le montre le `bonjour` affiché ci-dessus.

Remarque : Le pire des messages est celui qui n'apparaît pas. Un programme qui s'exécute sans erreur n'est pas pour autant un programme juste : il fait ce que vous avez écrit, non ce que vous vouliez. C'est pourquoi chaque programme de ce cours sera accompagné d'un résultat attendu, calculé à la main ou connu par ailleurs, auquel comparer l'affichage obtenu.

Ce que j'ai écrit	Ce que Python comprend
`print("Bonjour")`	affiche le texte `Bonjour`
`print(2 + 3)`	calcule, puis affiche `5`
`print("2 + 3")`	affiche le texte `2 + 3`
`# Bonjour`	n'affiche rien, ligne ignorée
`print(Bonjour)`	cherche une variable nommée `Bonjour`, et échoue

Ce tableau résume à lui seul le chapitre : tout l'art consiste à distinguer ce qui est du texte de ce qui est un calcul, et c'est l'objet du chapitre suivant, consacré aux variables et aux types.

Chapitre 2

Variables, types et calculs

Explicite vaut mieux qu'implicite.

Tim Peters - *The Zen of Python*, PEP 20, 2004

Un programme qui se contente d'afficher un texte fixe n'a guère d'intérêt : on l'aurait écrit plus vite à la main. Tout change dès que le programme manipule des valeurs qu'il ne connaît pas à l'avance, qu'il les range quelque part, les modifie, les combine. Ce chapitre installe les deux notions sur lesquelles reposera tout le reste du cours : la *variable*, qui retient une valeur, et le *type*, qui dit ce que l'on a le droit de faire avec.

2.1 Variables et affectation

2.1.1 Ranger une valeur

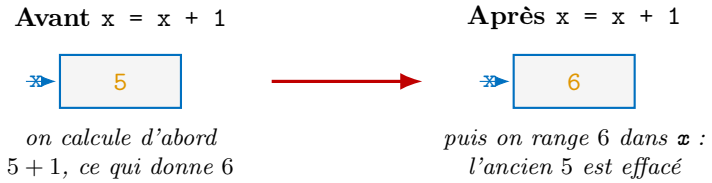
> **Définition :** Une **variable** est un nom associé à un emplacement de la mémoire, dans lequel est rangée une valeur. L'instruction `nom = valeur` s'appelle une **affectation** : elle range la valeur de droite dans la variable de gauche.

```
prix = 12.5
quantite = 4
total = prix * quantite
print(total)
```

Ce programme affiche `50.0`. L'interpréteur a procédé en quatre temps, dans l'ordre des lignes : il a rangé `12.5` sous le nom `prix`, puis `4` sous le nom `quantite`, puis il a calculé le produit `12.5 * 4` et rangé le résultat sous le nom `total`, et enfin il a affiché ce dernier.

Propriété : L'exécution d'une affectation `nom = expression` se fait toujours en deux étapes, **dans cet ordre** :

1. l'expression de droite est entièrement évaluée, c'est-à-dire calculée ;
2. la valeur obtenue est rangée dans la variable de gauche, dont l'ancien contenu est perdu.



Cette figure montre ce qui se passe dans la mémoire lors de l'instruction `x = x + 1` : l'étiquette `x` désigne toujours la même case, mais son contenu a changé.

Remarque : L'erreur la plus répandue est de lire `x = x + 1` comme une équation. Ce serait une équation sans solution, et pourtant l'instruction fonctionne parfaitement. Le signe `=` n'est pas ici le symbole d'une égalité, mais l'ordre « range à gauche ce que tu viens de calculer à droite ». Nous verrons au chapitre 3 que l'égalité mathématique, elle, se note `==`.

Remarque : Une affectation ne crée aucun lien durable entre les variables. Si l'on écrit `a = 3` puis `b = a` puis `a = 10`, la variable `b` vaut toujours 3 : elle a reçu une copie de la valeur au moment du passage de la ligne, et non une promesse de suivre `a`. Nous nuancerons ce point au chapitre 7, où les listes se comporteront différemment.

2.1.2 Nommer ses variables

Un nom de variable commence par une lettre ou le caractère `_`, ne contient ni espace ni accent ni signe de ponctuation, et ne doit pas être un mot réservé du langage comme `if`, `for`, `while` ou `print`. Python distingue les majuscules des minuscules : `Total` et `total` désignent deux variables différentes, ce qui est une source d'erreurs inépuisable.

♡ **Méthode :** Choisissez des noms qui disent ce que la variable contient. Le programme

```
n = 4
p = 12.5
t = n * p
```

et le programme

```
quantite = 4
prix_unitaire = 12.5
total = quantite * prix_unitaire
```

font exactement la même chose, mais seul le second se relit six mois plus tard. Réservez les noms d'une lettre aux objets mathématiques dont c'est l'usage : un indice `i`, un entier `n`, une abscisse `x`.

Exemple : Pour échanger le contenu de deux variables, Python offre une écriture particulièrement commode : l'instruction `a, b = b, a` évalue d'abord le couple de droite, puis affecte simultanément. Ainsi, après

```
a = 3
b = 7
a, b = b, a
```

la variable `a` vaut 7 et `b` vaut 3. Dans la plupart des autres langages, cet échange exige une variable auxiliaire ; nous nous en servirons au chapitre 8 pour écrire un algorithme de tri.

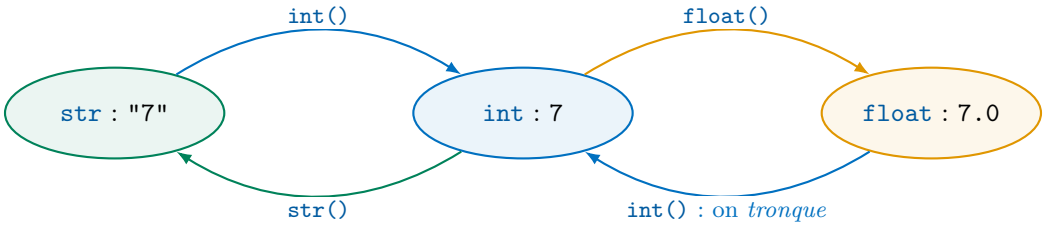
2.2 Les types de données

Définition : Le **type** d'une valeur est la nature de cette valeur. Les quatre types fondamentaux sont l'entier (`int`), le nombre à virgule flottante (`float`), la chaîne de caractères (`str`) et le booléen (`bool`). L'instruction `type(v)` renvoie le type de la valeur `v`.

Type	Nature	Exemples	Ensemble associé
`int`	entier relatif	`7`, `0`, `-15`	$\mathbb{Z}$
`float`	nombre décimal approché	`2.5`, `-0.75`, `3.0`	une partie de $\mathbb{D}$
`str`	texte	`"bonjour"`, `"12"`	—
`bool`	valeur de vérité	`True`, `False`	{vrai, faux}

Remarque : La virgule décimale s'écrit avec un *point* : 2,5 se code `2.5`. La virgule, en Python, sert exclusivement à séparer des éléments dans une liste ou des arguments dans des parenthèses. Écrire `x = 2,5` ne produit pas d'erreur, mais range dans `x` le couple $(2\,;5)$, ce qui fausse silencieusement tous les calculs suivants : c'est exactement le type d'erreur que Python ne signale pas.

2.2.1 Passer d'un type à l'autre



Chaque flèche de ce schéma correspond à une instruction de conversion ; on y lit en particulier que le passage de `float` à `int` n'est pas un arrondi, mais une troncature vers zéro. Les mêmes noms assurent le passage direct entre `str` et `float`.

Exemple : Les conversions les plus fréquentes donnent :

```
print(int(3.9))      # 3   : la partie décimale est supprimée
print(int(-3.9))     # -3  : on tronque vers 0, on n'arrondit pas
print(float(7))      # 7.0
print(str(7) + "1") # 71  : concaténation de deux textes
print(int("12") + 1)  # 13 : le texte "12" devient le nombre 12
```

La troisième ligne mérite attention : `str(7) + "1"` ne vaut pas 8 mais le texte `"71"`, car l'opérateur `+` appliqué à deux chaînes les met bout à bout. C'est ce qu'on appelle la **concaténation**.

Remarque : Pour obtenir un véritable arrondi, on dispose de `round`. Attention : `round(3.14159, 2)` donne `3.14`, mais `round(2.5)` donne `2` et non `3`. Python applique en effet l'arrondi *au pair le plus proche* en cas d'égalité parfaite, convention adoptée pour éviter un biais systématique lorsqu'on somme un grand nombre d'arrondis. La convention scolaire française, elle, arrondirait à 3.

Remarque : Un `float` n'est presque jamais la valeur exacte du nombre qu'il représente : c'est une approximation binaire. L'instruction `print(0.1 + 0.2)` affiche `0.30000000000000004`. Il n'y a là aucun bug : 0,1 n'a pas d'écriture binaire finie, de même que $\frac{1}{3}$ n'a pas d'écriture décimale finie. Nous en tirerons au chapitre 3 une conséquence pratique très importante sur la façon de comparer deux nombres.

2.3 Les opérations

Opérateur	Signification	Exemple	Résultat
`+ - *`	somme, différence, produit	`3 * 4`	`12`
`/`	quotient décimal	`7 / 2`	`3.5`
`**`	puissance	`2 ** 10`	`1024`
`//`	quotient entier	`7 // 2`	`3`
`%`	reste de la division	`7 % 2`	`1`

Les règles de priorité sont celles des mathématiques : la puissance d'abord, puis les produits et quotients, puis les sommes et différences, les parenthèses l'emportant sur tout. Ainsi `2 + 3 * 4` vaut 14, et il faut écrire `(2 + 3) * 4` pour obtenir 20.

Remarque : L'opérateur `/` renvoie *toujours* un `float`, même lorsque la division tombe juste : `10 / 2` vaut `5.0` et non `5`. Cela n'a aucune importance dans un calcul, mais en a une lorsqu'on affiche un résultat ou qu'on s'en sert comme indice au chapitre 7.

Propriété : Pour deux entiers a et b avec $b > 0$, les instructions `a // b` et `a % b` donnent respectivement le quotient q et le reste r de la **division euclidienne** de a par b, c'est-à-dire l'unique couple d'entiers tel que

$$a = b \times q + r \qquad \text{avec} \qquad 0 \leqslant r < b.$$

Exemple : On a `17 // 5` qui vaut 3 et `17 % 5` qui vaut 2, et l'on vérifie que $17 = 5 \times 3 + 2$ avec $0 \leqslant 2 < 5$. Le reste sert constamment : un nombre n est pair si et seulement si `n % 2` vaut 0, et divisible par 7 si et seulement si `n % 7` vaut 0. Nous nous en servirons dès le chapitre 3.

Remarque : Sur les entiers négatifs, Python reste fidèle à la définition mathématique ci-dessus, ce qui surprend : `-7 // 2` vaut -4, et non -3, car $-7 = 2 \times (-4) + 1$ avec un reste 1 bien compris entre 0 et 2. De même `-7 % 2` vaut 1. Beaucoup de calculatrices, elles, tronquent vers zéro et annoncent -3 : ce n'est pas la même convention, et c'est Python qui a raison du point de vue de l'arithmétique.

2.3.1 Le module `math`

Les fonctions mathématiques usuelles ne sont pas disponibles immédiatement : il faut charger le **module** qui les contient, au moyen d'une instruction `import` placée en tête du programme.

```
import math

print(math.sqrt(2))      # 1.4142135623730951
print(math.pi)           # 3.141592653589793
print(math.floor(3.7))   # 3, la partie entière
```

Remarque : La racine carrée obtenue est une valeur approchée à une quinzaine de décimales, jamais le nombre $\sqrt{2}$ lui-même, qui est irrationnel. Python calcule vite, mais il calcule de façon approchée dès qu'il quitte les entiers : il ne remplacera donc jamais une démonstration.

2.4 Dialoguer avec l'utilisateur

> **Définition :** L'instruction `input("message")` affiche le message, attend que l'utilisateur tape quelque chose puis valide, et renvoie ce qui a été tapé **sous forme de chaîne de caractères**.

```
prenom = input("Quel est ton prénom ? ")
print("Bonjour", prenom)
```

♡ **Méthode :** Pour récupérer un *nombre*, il faut convertir explicitement ce que renvoie `input`, car c'est toujours du texte.

```
age = int(input("Quel est ton âge ? "))
print("L'an prochain tu auras", age + 1, "ans")
```

Sans le `int(...)`, l'instruction `age + 1` provoquerait l'erreur `TypeError` rencontrée au chapitre 1, puisqu'on ne peut pas ajouter 1 à un texte.

Remarque : Cette conversion est aussi un piège : si l'utilisateur tape `douze` au lieu de `12`, l'instruction `int(...)` échoue avec une `ValueError`. Un programme robuste devrait prévoir ce cas ; nous en aurons les moyens au chapitre suivant, avec les tests.

Remarque : Dans les exercices de mathématiques, on préfère souvent se passer de `input` et fixer les valeurs en tête de programme, ou mieux, les passer en paramètres d'une fonction, comme nous le ferons au chapitre 6. Un programme sans `input` a l'immense avantage de se relancer à l'identique, ce qui permet de comparer deux exécutions.

Chapitre 3

Tests et conditions

Les tests révèlent la présence de bugs, non leur absence.

Edsger W. Dijkstra - *Software Engineering Techniques*, conférence de l'OTAN, Rome, 1969, p. 16

Jusqu'ici, nos programmes s'exécutaient ligne après ligne, toujours de la même façon. Ils ne savaient pas *choisir*. Or la plupart des raisonnements mathématiques procèdent par disjonction de cas : si le discriminant est négatif alors il n'y a pas de solution réelle, sinon il y en a. Ce chapitre donne à nos programmes ce pouvoir de bifurquer, et il commence par l'objet qui le rend possible : le booléen.

3.1 Booléens et comparaisons

> **Définition :** Un **booléen** est une valeur de vérité. Il n'en existe que deux, notées `True` et `False`, avec une majuscule. Le type correspondant est `bool`.

Les booléens ne se tapent presque jamais directement : ils apparaissent comme *résultat* d'une comparaison. L'instruction `print(3 < 5)` affiche `True`, car l'affirmation « 3 est strictement inférieur à 5 » est vraie.

En Python	Signification	En Python	Signification
`a == b`	a est égal à b	`a != b`	a est différent de b
`a < b`	$a < b$	`a <= b`	$a \leqslant b$
`a > b`	$a > b$	`a >= b`	$a \geqslant b$

Remarque : Le double signe `==` n'est pas une coquetterie. Le simple `=` est déjà pris : il sert à affecter, comme nous l'avons vu au chapitre 2. Confondre les deux est l'erreur la plus répandue du débutant, et Python la signale par une `SyntaxError` : écrire `if x = 3` n'a aucun sens, puisqu'on ne peut pas « ranger dans un test ». Retenez la formule : un seul signe pour *ranger*, deux signes pour *demander*.

Remarque : On peut comparer autre chose que des nombres. Les chaînes se comparent dans l'ordre du dictionnaire, si bien que `"abricot" < "banane"` vaut `True`. Attention toutefois : les majuscules passent avant toutes les minuscules, donc `"Zoé" < "abricot"` vaut également `True`.

♡ **Méthode :** Ne testez jamais l'égalité de deux nombres à virgule par `==`. Nous avons vu au chapitre 2 que `0.1 + 0.2` vaut `0.30000000000000004` : l'instruction `0.1 + 0.2 == 0.3` renvoie donc `False`. On teste à la place que l'écart est négligeable :

```
if abs(x - y) < 1e-9:
    print("x et y sont égaux, aux erreurs d'arrondi près")
```

La notation `1e-9` désigne 10^{-9}. Cette précaution ne concerne que les `float` : sur des entiers, `==` est parfaitement fiable.

3.2 Combiner des conditions

Définition : Étant donné deux booléens A et B, l'expression `A and B` est vraie lorsque les deux le sont, l'expression `A or B` est vraie lorsque l'un au moins l'est, et `not A` est vraie exactement quand A est fausse.

A	B	`A and B`	`A or B`	`not A`
`True`	`True`	`True`	`True`	`False`
`True`	`False`	`False`	`True`	`False`
`False`	`True`	`False`	`True`	`True`
`False`	`False`	`False`	`False`	`True`

Cette table de vérité est celle de la logique enseignée en mathématiques : le `and` est la conjonction « et », le `or` la disjonction « ou », toujours inclusive.

Exemple : Pour exprimer que x appartient à l'intervalle $[2\,;5]$, on écrit `x >= 2 and x <= 5`. Python autorise d'ailleurs l'écriture mathématique condensée `2 <= x <= 5`, qui donne le même résultat et se lit mieux. En revanche, pour exprimer que x est en dehors de cet intervalle, il faut un `or` : `x < 2 or x > 5`.

Remarque : Le passage d'une condition à sa contraire échange systématiquement `and` et `or` : la négation de « $x \geqslant 2$ *et* $x \leqslant 5$ » est « $x < 2$ *ou* $x > 5$ ». Ce sont les lois de De Morgan, et l'oubli de cet échange est une cause classique de programmes qui semblent corrects et ne le sont pas.

3.3 L'instruction conditionnelle

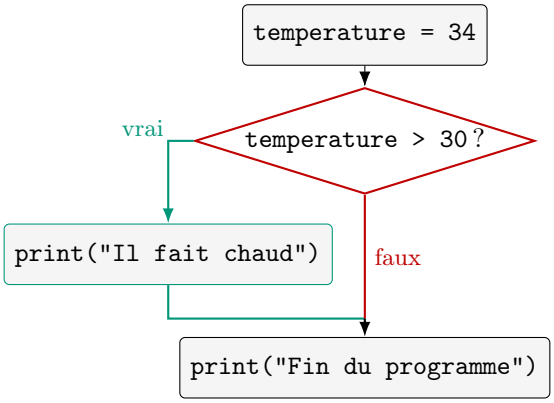
3.3.1 La forme simple

Syntaxe : L'instruction `if` exécute un bloc d'instructions si et seulement si la condition est vraie.

```
if temperature > 30:
    print("Il fait chaud")
    print("Pensez à boire")
print("Fin du programme")
```

Deux éléments sont obligatoires : le **deux-points** à la fin de la ligne `if`, et l'**indentation** des lignes du bloc, c'est-à-dire leur décalage vers la droite de quatre espaces.

Dans l'exemple ci-dessus, les deux premiers `print` sont décalés : ils forment le bloc soumis à la condition. Le troisième ne l'est pas : il s'exécute dans tous les cas. L'indentation n'est donc pas une question de présentation, c'est elle qui *définit* l'étendue du bloc. Python est l'un des rares langages où la mise en page a valeur d'instruction.

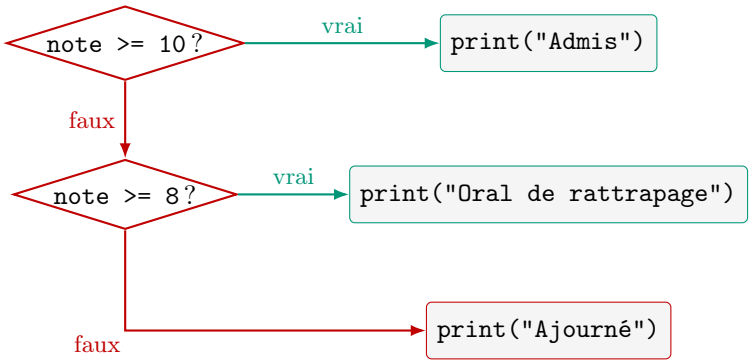


On lit sur cet organigramme que les deux chemins se rejoignent : quelle que soit la température, la dernière instruction est exécutée.

3.3.2 Les formes `else` et `elif`

Syntaxe : Le mot `else` introduit le bloc exécuté lorsque la condition est fausse ; le mot `elif`, contraction de *else if*, introduit une condition supplémentaire examinée seulement si les précédentes ont échoué.

```
if note >= 10:
    print("Admis")
elif note >= 8:
    print("Oral de rattrapage")
else:
    print("Ajourné")
```



L'essentiel est dans la descente verticale : on ne passe au test suivant que si le précédent a échoué, et une seule des trois branches est exécutée.

Remarque : L'ordre des tests est donc capital. Si l'on inverse les deux premières conditions et qu'on écrit `if note >= 8` en premier, un élève ayant 15 obtient « Oral de rattrapage » : sa note vérifie bien la première condition rencontrée, et Python s'arrête là. L'erreur la plus répandue dans les cascades de `elif` consiste à placer la condition la plus large avant la plus restrictive.

Remarque : Une suite de `if` indépendants n'est pas équivalente à une cascade de `elif` : avec quatre `if` successifs, les quatre conditions sont examinées et plusieurs blocs peuvent s'exécuter. Utilisez `elif` exactement quand les cas s'excluent mutuellement, ce qui est le cas de la plupart des disjonctions de cas mathématiques.

3.4 Exemples rédigés

Exemple : **Signe d'un nombre.** La disjonction de cas comporte trois branches exclusives, ce qui appelle un `elif` :

```
x = -4
if x > 0:
    print("positif")
elif x < 0:
    print("négatif")
else:
    print("nul")
```

Avec $x = -4$, la première condition est fausse, la seconde vraie : le programme affiche `négatif`. Remarquez que le dernier cas ne demande aucun test, puisqu'il ne reste que la possibilité $x = 0$.

Exemple : **Résolution d'une équation du second degré.** Le programme suivant traduit exactement le théorème du cours de première.

```
import math

a, b, c = 1, -5, 6
```

```
delta = b**2 - 4*a*c

if delta < 0:
    print("Aucune solution réelle")
elif delta == 0:
    print("Une solution double :", -b / (2*a))
else:
    x1 = (-b - math.sqrt(delta)) / (2*a)
    x2 = (-b + math.sqrt(delta)) / (2*a)
    print("Deux solutions :", x1, "et", x2)
```

Ici $\Delta = 25 - 24 = 1$, et le programme affiche `Deux solutions : 2.0 et 3.0`. On retrouve bien les racines de $x^2 - 5x + 6 = (x-2)(x-3)$. Notez que le test `delta == 0` est ici légitime, car a, b et c sont des entiers : Δ est donc calculé exactement. Si les coefficients étaient des `float`, il faudrait écrire `abs(delta) < 1e-9`.

Exemple : **Année bissextile.** Une année est bissextile si elle est divisible par 4, sauf si elle est divisible par 100 sans l'être par 400. Cette règle se traduit avec le reste `%` du chapitre 2 :

```
annee = 2024
if annee % 400 == 0:
    bissextile = True
elif annee % 100 == 0:
    bissextile = False
elif annee % 4 == 0:
    bissextile = True
else:
    bissextile = False
print(annee, "est bissextile :", bissextile)
```

Le programme affiche `2024 est bissextile : True`. On vérifie qu'il répond `False` pour 1900, divisible par 100 mais pas par 400, et `True` pour 2000. L'ordre des trois tests est ici l'exacte transcription de la hiérarchie des exceptions : c'est un cas où permuter deux lignes donnerait un programme faux sans provoquer la moindre erreur.

Remarque : Le programme précédent peut s'écrire en une ligne, en affectant directement le résultat de l'expression booléenne :

```
bissextile = (annee % 4 == 0 and annee % 100 != 0) or annee % 400 == 0
```

C'est plus court, mais plus difficile à relire et à corriger. Tant que vous débutez, préférez la version en cascade : un programme juste et long vaut mieux qu'un programme court et douteux.

Chapitre 4

Boucles bornées : l'instruction `for`

Un algorithme doit être vu pour être cru, et la meilleure façon d'apprendre ce qu'il fait est de l'essayer.

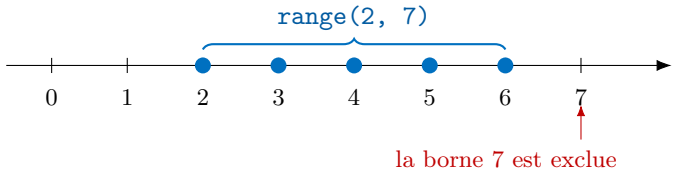
Donald E. Knuth - *The Art of Computer Programming*, volume 1, section 1.1, 1968

Un ordinateur n'est pas plus intelligent qu'un élève de sixième, mais il ne se lasse jamais. C'est cette endurance que nous allons exploiter : répéter une opération mille fois lui coûte le même effort que la répéter une fois. Ce chapitre traite du cas où l'on sait *à l'avance* combien de répétitions sont nécessaires ; le chapitre suivant traitera du cas contraire.

4.1 Parcourir une plage d'entiers

Définition : L'instruction `range(n)` désigne la suite des entiers $0, 1, 2, \ldots, n-1$. Plus généralement, `range(a, b)` désigne les entiers de a à $b-1$, et `range(a, b, p)` ceux que l'on obtient en partant de a et en avançant de p en p, sans jamais atteindre b.

Instruction	Entiers parcourus	Nombre de valeurs
`range(5)`	0, 1, 2, 3, 4	5
`range(2, 7)`	2, 3, 4, 5, 6	5
`range(0, 10, 3)`	0, 3, 6, 9	4
`range(10, 0, -2)`	10, 8, 6, 4, 2	5
`range(1, n+1)`	1, 2, ..., n	n



La figure rappelle la convention essentielle : la première borne est atteinte, la seconde ne l'est jamais. On retrouve la logique des intervalles semi-ouverts $[a\,;b[$ du cours de mathématiques, et le nombre d'entiers parcourus vaut exactement $b-a$.

Remarque : L'erreur la plus répandue est d'oublier cette exclusion. Pour parcourir les entiers de 1 à n *inclus*, il faut écrire `range(1, n+1)` et non `range(1, n)`. Un programme qui donne un résultat presque juste, à un terme près, souffre presque toujours de ce défaut-là.

4.2 La boucle `for`

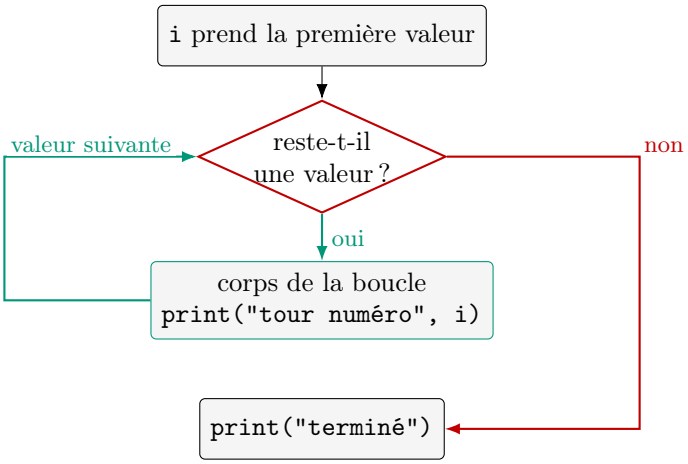
Syntaxe : La boucle `for` exécute un bloc d'instructions une fois pour chaque valeur parcourue.

```
for i in range(4):
    print("tour numéro", i)
print("terminé")
```

Comme pour le `if` du chapitre 3, le deux-points et l'indentation délimitent le bloc répété, appelé le **corps** de la boucle.

Affichage obtenu :

```
tour numéro 0
tour numéro 1
tour numéro 2
tour numéro 3
terminé
```



Cet organigramme montre le mécanisme complet : la variable de boucle prend successivement chacune des valeurs, le corps est exécuté une fois par valeur, et l'on ne sort que lorsque la plage est épuisée.

Remarque : La variable de boucle est mise à jour automatiquement à chaque tour. La modifier à l'intérieur du corps ne sert donc à rien : écrire `i = i + 10` dans le corps ne fait pas sauter d'étapes, car la valeur est écrasée au tour suivant. C'est une différence importante avec la boucle `while` du chapitre 5, où c'est au programmeur de gérer l'avancement.

Remarque : Si la variable de boucle ne sert pas dans le corps — par exemple pour répéter dix fois le même affichage — l'usage est de la nommer `_`, ce qui signale au lecteur qu'elle est délibérément ignorée.

4.3 Accumuler un résultat

C'est l'usage le plus fréquent de la boucle : faire grossir progressivement une variable qui contiendra, à la sortie, le résultat cherché.

♡ **Méthode :** Pour calculer une somme, un produit ou un compte, on procède en trois temps.

1. **Initialiser** l'accumulateur *avant* la boucle : à 0 pour une somme, à 1 pour un produit.
2. **Mettre à jour** l'accumulateur dans le corps de la boucle.
3. **Exploiter** l'accumulateur *après* la boucle, une fois celle-ci terminée.

```
S = 0
for i in range(1, 101):
    S = S + i
print("La somme des entiers de 1 à 100 vaut", S)
```

Le programme affiche 5050, ce que confirme la formule $\frac{n(n+1)}{2} = \frac{100\times101}{2} = 5050$ établie en classe de première. Le tableau ci-dessous, appelé **tableau de trace**, suit l'état des variables aux premiers tours ; c'est l'outil à réflexe quand un programme ne donne pas ce qu'on attend.

Tour	avant	1er	2e	3e	4e	...	100e
i	—	1	2	3	4	...	100
S	0	1	3	6	10	...	5050

Remarque : L'initialisation doit impérativement précéder la boucle. Si l'on écrit `S = 0` *dans* le corps, l'accumulateur est remis à zéro à chaque tour et le programme affiche 100, la valeur du dernier terme. Si on l'oublie tout à fait, Python signale une `NameError` au premier passage.

Exemple : Le même schéma, avec un produit, calcule une factorielle. On initialise cette fois à 1, élément neutre de la multiplication :

```
P = 1
for k in range(1, 6):
    P = P * k
print("5! =", P)
```

Le programme affiche `5! = 120`, et l'on vérifie à la main que $1\times2\times3\times4\times5 = 120$. Initialiser P à 0 donnerait 0 : dans une accumulation, la valeur de départ doit être l'élément neutre de l'opération employée.

Exemple : Pour compter les multiples de 7 inférieurs à 100, on combine la boucle du présent chapitre et le test du chapitre 3 :

```
compteur = 0
for n in range(1, 100):
    if n % 7 == 0:
        compteur = compteur + 1
print(compteur)
```

Le programme affiche 14, ce qui se contrôle sans machine : les multiples de 7 strictement compris entre 0 et 100 vont de 7×1 à $7\times14 = 98$. Notez la double indentation : le `if` est dans la boucle, et l'incrémentation est dans le `if`.

4.4 Calculer les termes d'une suite

Une suite définie par récurrence se prête admirablement à la boucle `for` : la relation $u_{n+1} = f(u_n)$ *est* l'instruction de mise à jour.

Exemple : Soit la suite définie par $u_0 = 2$ et $u_{n+1} = 0{,}8\,u_n + 3$. Le programme suivant affiche les six premiers termes.

```
u = 2
print("u0 =", u)
for n in range(1, 6):
    u = 0.8 * u + 3
    print("u" + str(n), "=", round(u, 4))
```

Affichage obtenu :

```
u0 = 2
u1 = 4.6
u2 = 6.68
u3 = 8.344
u4 = 9.6752
u5 = 10.7402
```

Les termes croissent en se rapprochant de 15, qui est la solution de l'équation $\ell = 0{,}8\,\ell + 3$: le programme suggère la limite, il ne la démontre pas. C'est là tout l'intérêt et toute la limite de la machine, et nous y reviendrons au chapitre 5 en cherchant à partir de quel rang la suite dépasse un seuil donné.

Remarque : Une seule variable `u` suffit : chaque tour écrase l'ancien terme par le nouveau. Si l'on souhaite conserver *tous* les termes pour les tracer ou les réutiliser, il faudra la structure de données du chapitre 7, la liste.

4.5 Boucles imbriquées

Le corps d'une boucle peut contenir une autre boucle. La boucle intérieure effectue alors un tour complet pour chaque valeur de la boucle extérieure.

```
for i in range(1, 4):
    for j in range(1, 4):
        print(i, "x", j, "=", i * j)
    print()
```

Ce programme affiche les neuf produits de la table, groupés en trois blocs séparés par une ligne vide, car l'instruction `print()` est indentée au niveau de la boucle extérieure seulement. Au total, le corps intérieur est exécuté $3 \times 3 = 9$ fois.

Remarque : Le nombre de répétitions se multiplie : deux boucles imbriquées parcourant chacune 1000 valeurs exécutent un million de tours. C'est encore instantané, mais trois boucles de 1000 feraient un milliard de tours et plusieurs minutes d'attente. Garder en tête ce décompte est le début de ce qu'on appelle l'étude de la complexité d'un algorithme.

Remarque : Les deux boucles doivent utiliser des variables différentes. Réutiliser `i` à l'intérieur écraserait le compteur extérieur et produirait un comportement inattendu : Python, fidèle à lui-même, ne protesterait pas.

Chapitre 5

Boucles non bornées : l'instruction `while`

Face à l'ambiguïté, résistez à la tentation de deviner.

Tim Peters - *The Zen of Python*, PEP 20, 2004

Au chapitre précédent, le nombre de tours était connu dès l'écriture du programme. Mais que faire pour répondre à la question « à partir de quel rang la suite dépasse-t-elle 1000 ? » Personne ne connaît la réponse avant d'avoir calculé, et c'est précisément la réponse qu'on cherche. Il faut donc une boucle qui s'arrête non pas après un nombre fixé de tours, mais dès qu'une condition cesse d'être vraie.

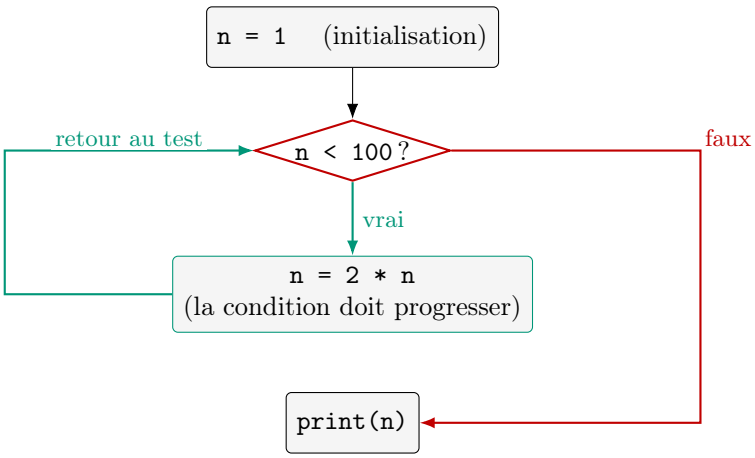
5.1 Répéter tant qu'une condition est vraie

Syntaxe : La boucle `while` répète son corps aussi longtemps que la condition reste vraie.

```
n = 1
while n < 100:
    n = 2 * n
print(n)
```

La condition est testée **avant** chaque tour, y compris le premier : si elle est fausse d'emblée, le corps n'est jamais exécuté.

Ce programme affiche 128. En effet `n` prend successivement les valeurs $1, 2, 4, 8, 16, 32, 64$, puis 128 ; à ce moment la condition `n < 100` devient fausse et la boucle s'arrête. Notez que la valeur finale dépasse 100 : la boucle ne s'interrompt pas au milieu d'un tour, elle termine toujours le corps commencé.



La comparaison avec l'organigramme du chapitre 4 est instructive : la forme est la même, mais l'avancement n'est plus automatique. C'est le corps de la boucle qui doit, lui-même, rapprocher la condition de sa fausseté.

Propriété : Une boucle `while` s'arrête si, et seulement si, la condition finit par devenir fausse. Pour s'en assurer, on vérifie trois points :

1. une variable de la condition est **initialisée avant** la boucle ;
2. cette variable est **modifiée dans** le corps ;
3. cette modification fait **progresser** la condition vers la fausseté.

Remarque : Si l'un des trois points manque, on obtient une **boucle infinie** : le programme tourne indéfiniment, sans message d'erreur, et il faut l'interrompre par la combinaison de touches `Ctrl + C`. L'oubli le plus fréquent est celui de la ligne d'avancement dans le corps : la condition, alors, reste éternellement vraie. Ce n'est pas une catastrophe, c'est le rite d'initiation de tout programmeur.

Remarque : La boucle infinie n'est pas toujours une étourderie. Avec une suite définie par $u_{n+1} = \frac{u_n}{2}$ et la condition `u > 0`, la boucle ne s'arrête jamais en mathématiques, mais elle s'arrête en Python : les `float` finissent par atteindre exactement 0 par manque de précision. Un programme peut donc terminer alors que l'algorithme correspondant ne termine pas, et c'est une raison de plus de ne jamais confondre les deux.

5.2 Rechercher un seuil

C'est l'application reine de la boucle `while` et un classique du baccalauréat : déterminer le premier rang à partir duquel une suite franchit une valeur donnée.

Exemple : **Suite géométrique décroissante.** Cherchons le plus petit entier n tel que $0{,}9^n < 0{,}01$.

```
n = 0
v = 1.0          # v contient 0.9 puissance n
while v >= 0.01:
    v = v * 0.9
    n = n + 1
print("n =", n, "et 0.9^n =", v)
```

Le programme affiche `n = 44 et 0.9^n = 0.009697737297875247`. Le résultat s'interprète ainsi : $0{,}9^{43} \approx 0{,}0108$ est encore supérieur au seuil, tandis que $0{,}9^{44} \approx 0{,}0097$ lui est inférieur. On peut le confirmer par le calcul, puisque l'inégalité $0{,}9^n < 0{,}01$ équivaut à $n > \frac{\ln(0{,}01)}{\ln(0{,}9)} \approx 43{,}7$.

Remarque : L'ordre des deux lignes du corps n'a aucune importance ici, mais leur présence conjointe est essentielle : `v` fait avancer la condition, `n` compte les tours. Une erreur classique consiste à incrémenter `n` avant la boucle ou après le `print`, ce qui décale le résultat d'une unité. Le tableau de trace du chapitre 4 reste le meilleur remède.

Exemple : **Placement à intérêts composés.** Un capital de 1200 euros augmente de 3 % par an. Au bout de combien d'années dépasse-t-il 1500 euros ?

```
capital = 1200
annees = 0
while capital < 1500:
    capital = capital * 1.03
    annees = annees + 1
print(annees, "années, capital =", round(capital, 2))
```

L'affichage est `8 années, capital = 1520.12`. Au bout de sept ans le capital n'atteignait que 1475,85 euros : il faut donc bien attendre la huitième année. On notera que la condition porte sur le capital, mais que la réponse attendue est le compteur : c'est presque toujours le cas dans ce type d'exercice.

5.3 `for` ou `while` ?

Boucle `for`	**Boucle `while`**
Le nombre de tours est connu avant l'exécution.	Le nombre de tours est inconnu, il dépend des calculs.
L'avancement est automatique.	L'avancement est à la charge du programmeur.
Elle se termine toujours.	Elle peut ne jamais se terminer.
Typique : calculer u_{10}, sommer n termes.	Typique : chercher un seuil, un pgcd, une racine.

Remarque : Toute boucle `for` peut se réécrire avec un `while`, mais l'inverse est faux, et de toute façon inutile : quand le nombre de tours est connu, le `for` dit mieux ce qu'il fait et supprime tout risque de boucle infinie. Choisissez la boucle qui exprime votre intention, pas celle que vous maîtrisez le mieux.

5.4 Deux algorithmes d'arithmétique

Exemple : **Division euclidienne par soustractions successives.** Avant de disposer des opérateurs `//` et `%` du chapitre 2, on peut retrouver quotient et reste en soustrayant b autant de fois que possible.

```
a, b = 47, 6
q = 0
r = a
while r >= b:
    r = r - b
    q = q + 1
print(a, "=", b, "x", q, "+", r)
```

Le programme affiche `47 = 6 x 7 + 5`. La condition d'arrêt `r >= b` est exactement la condition $0 \leqslant r < b$ de la définition mathématique : la boucle s'arrête au moment précis où le reste devient légitime. À chaque tour, la quantité r diminue strictement de b, ce qui garantit la terminaison.

Exemple : **Algorithme d'Euclide.** Le plus grand diviseur commun de deux entiers s'obtient en remplaçant répétitivement le couple $(a\,;b)$ par $(b\,;r)$, où r est le reste de a par b, jusqu'à obtenir un reste nul.

```
a, b = 1071, 462
while b != 0:
    a, b = b, a % b
print("pgcd =", a)
```

Tour	1er	2e	3e	sortie
`a`	1071	462	147	**21**
`b`	462	147	21	0
`a % b`	147	21	0	—

Le tableau de trace se lit colonne par colonne : chaque nouveau couple est formé de l'ancien `b` et de l'ancien reste. La boucle s'arrête lorsque `b` vaut 0, et c'est alors `a` qui contient le pgcd, ici 21. On vérifie que $1071 = 3^2 \times 7 \times 17$ et $462 = 2 \times 3 \times 7 \times 11$, dont le pgcd est bien $3 \times 7 = 21$.

Remarque : L'affectation simultanée `a, b = b, a % b` est indispensable. En deux instructions séparées, `a = b` puis `b = a % b`, la seconde ligne utiliserait le *nouveau* `a`, et calculerait `b % b`, c'est-à-dire 0 : la boucle s'arrêterait dès le premier tour en annonçant un pgcd faux. Le mécanisme d'échange vu au chapitre 2 prend ici tout son sens.

Remarque : Cet algorithme figure déjà, sous forme géométrique, dans les *Éléments* d'Euclide, au IIIe siècle avant notre ère. Il est d'une efficacité remarquable : quelques dizaines de tours suffisent pour des nombres de vingt chiffres, là où la recherche systématique des diviseurs serait hors de portée de toute machine. Nous retrouverons cette idée de réduction rapide au chapitre 8 avec la dichotomie.

Chapitre 6

Les fonctions

N'importe quel imbécile peut écrire du code qu'un ordinateur comprend. Les bons programmeurs écrivent du code que les humains comprennent.

Martin Fowler - *Refactoring : Improving the Design of Existing Code*, 1999, p. 15

Nos programmes commencent à se répéter. Le calcul du discriminant du chapitre 3, la recherche de seuil du chapitre 5 : chaque fois qu'on veut les refaire avec d'autres nombres, il faut recopier les lignes et modifier les valeurs à la main. La fonction résout ce problème, et elle en résout un second, plus profond : elle permet de donner un *nom* à une idée, et donc de raisonner avec des briques plutôt qu'avec des lignes.

6.1 Définir et appeler une fonction

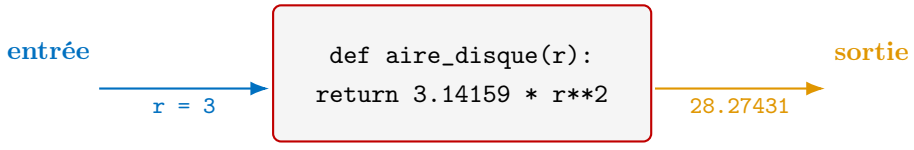
> **Définition :** Une **fonction** est un bloc d'instructions auquel on a donné un nom, et qui peut recevoir des **paramètres** et renvoyer un résultat. On la définit par le mot-clé `def`, et on l'exécute en l'**appelant** par son nom suivi de parenthèses.

Syntaxe : La définition annonce le nom et les paramètres ; l'instruction `return` fixe le résultat renvoyé.

```
def aire_disque(r):
    return 3.14159 * r**2

print(aire_disque(3))
print(aire_disque(10))
```

Ce programme affiche `28.27431` puis `314.159`. La définition, à elle seule, ne calcule rien : elle se contente d'enregistrer la recette. C'est l'appel `aire_disque(3)` qui déclenche le calcul, en donnant temporairement au paramètre `r` la valeur 3.



La fonction se comporte exactement comme une fonction mathématique : à une entrée elle associe une sortie, et l'on peut s'en servir sans savoir ce qu'il y a dans la boîte. C'est d'ailleurs ce que nous faisons depuis le chapitre 2 avec `math.sqrt`, dont nous ignorons tout du contenu.

> *Remarque :* Les parenthèses sont obligatoires à l'appel, même sans paramètre. Écrire le nom seul ne provoque pas d'erreur : Python affiche alors la fonction elle-même, sous une forme énigmatique commençant par `<function`, et non son résultat. C'est déroutant la première fois, mais parfaitement logique.

6.2 return n'est pas print

C'est la difficulté majeure du chapitre, et elle vaut la peine qu'on s'y arrête. Comparons deux fonctions apparemment jumelles.

```
def double_affiche(x):
    print(2 * x)

def double_renvoie(x):
    return 2 * x

a = double_affiche(5)      # affiche 10, puis a vaut None
b = double_renvoie(5)      # n'affiche rien, mais b vaut 10
print(a)
print(b)
```

Affichage obtenu :

```
10
None
10
```

Propriété : L'instruction `print` s'adresse à l'**utilisateur** : elle dépose un texte à l'écran et rien de plus. L'instruction `return` s'adresse au **programme** : elle fournit une valeur réutilisable dans un calcul. Une fonction sans `return` renvoie automatiquement la valeur spéciale `None`, qui signifie « rien ».

Remarque : L'erreur la plus répandue consiste à écrire `print` au lieu de `return`, puis à s'étonner que `2 * double_affiche(5)` déclenche une `TypeError`. On ne peut effectivement pas multiplier « rien » par 2. Retenez la règle : une fonction qui *calcule* doit renvoyer ; c'est le programme principal qui affiche.

Remarque : L'exécution de `return` met fin immédiatement à la fonction : les instructions qui suivent dans le bloc ne sont jamais exécutées. C'est commode pour traiter un cas particulier au début d'une fonction et sortir aussitôt, comme dans l'exemple des solutions ci-dessous.

6.3 Plusieurs paramètres, plusieurs résultats

Une fonction peut recevoir autant de paramètres qu'on le souhaite, séparés par des virgules, et renvoyer plusieurs valeurs à la fois.

Exemple : Reprenons l'équation du second degré du chapitre 3, cette fois sous forme de fonction. Elle renvoie soit `None`, soit un nombre, soit un couple de nombres.

```
import math

def solutions(a, b, c):
    """Renvoie les solutions réelles de a*x**2 + b*x + c = 0."""
    delta = b**2 - 4*a*c
    if delta < 0:
        return None
    if delta == 0:
        return -b / (2*a)
    x1 = (-b - math.sqrt(delta)) / (2*a)
    x2 = (-b + math.sqrt(delta)) / (2*a)
    return x1, x2

print(solutions(1, -5, 6))     # (2.0, 3.0)
print(solutions(1, 2, 1))      # -1.0
print(solutions(2, 3, 5))      # None
```

Les trois appels correspondent aux trois cas du théorème : $\Delta = 1 > 0$, puis $\Delta = 0$, puis $\Delta = -31 < 0$. Le programme principal tient désormais en trois lignes, et le raisonnement

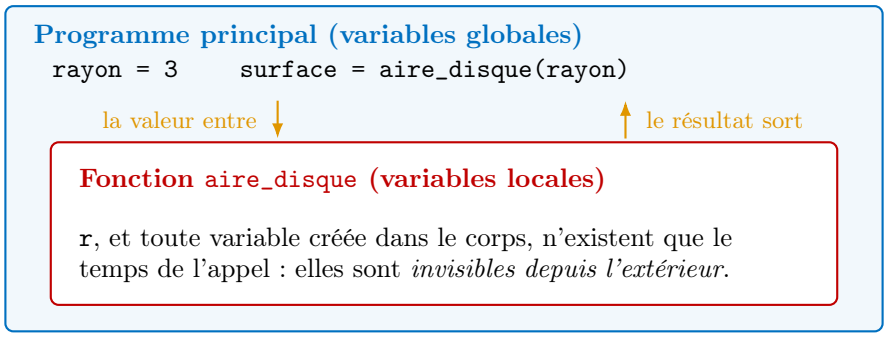
mathématique est isolé dans la fonction, où on peut le relire et le corriger une bonne fois pour toutes.

Remarque : Le texte placé entre triples guillemets juste après la ligne `def` s'appelle une **docstring**. Il n'est pas exécuté, mais il constitue la documentation de la fonction : l'instruction `help(solutions)` l'affiche. Prenez l'habitude d'y écrire ce que la fonction reçoit et ce qu'elle renvoie, en une phrase.

Remarque : On peut donner à un paramètre une valeur par défaut, utilisée lorsque l'appel ne la précise pas. Avec `def arrondi(x, n=2)`, l'appel `arrondi(3.14159)` utilise $n = 2$ tandis que `arrondi(3.14159, 4)` impose $n = 4$. Les paramètres munis d'une valeur par défaut doivent figurer en dernier dans la liste.

6.4 La portée des variables

> **Définition :** Une variable créée à l'intérieur d'une fonction est dite **locale** : elle n'existe que pendant l'exécution de cette fonction et disparaît ensuite. Une variable créée dans le programme principal est dite **globale**.



Le schéma se lit ainsi : la fonction est une pièce fermée ; seules les valeurs des paramètres y entrent, seul le `return` en sort. Ce cloisonnement est une protection, non une contrainte : il garantit qu'une fonction écrite aujourd'hui ne viendra pas perturber un programme écrit demain.

Exemple : Le programme suivant illustre la disparition des variables locales.

```
def calcul(n):
    resultat = n * n
    return resultat

print(calcul(4))       # affiche 16
print(resultat)        # NameError : 'resultat' is not defined
```

La dernière ligne échoue, bien que la variable ait manifestement existé : elle a été détruite à la sortie de la fonction. Pour conserver le résultat, il faut le recueillir dans une variable du programme principal, par exemple `valeur = calcul(4)`.

Remarque : À l'inverse, une fonction peut *lire* une variable globale. C'est possible, mais fortement déconseillé : la fonction devient dépendante du contexte, et donne des résultats différents selon l'endroit d'où on l'appelle. Une bonne fonction reçoit tout ce dont elle a besoin par ses paramètres.

6.5 Réutiliser des fonctions existantes

Les fonctions les plus utiles sont souvent déjà écrites. Certaines sont disponibles d'emblée, comme `print`, `len`, `abs`, `round`, `min` et `max`. D'autres résident dans des modules qu'il faut importer, comme `math` rencontré au chapitre 2.

```
import random

de = random.randint(1, 6)        # un entier au hasard entre 1 et 6 inclus
piece = random.random()          # un décimal au hasard dans [0 ; 1[
```

Remarque : La fonction `random.randint(a, b)` est l'une des rares de Python dont *les deux bornes sont incluses*, contrairement à `range` du chapitre 4. Confondre les deux conventions fausse toutes les simulations : un dé simulé par `random.randint(1, 6)` est équilibré, un dé simulé par `random.randint(0, 6)` a sept faces.

Exemple : Combiner une fonction et une boucle donne des programmes très courts. Voici le calcul de la moyenne de vingt lancers d'un dé :

```
import random

def lancer_de():
    return random.randint(1, 6)

total = 0
for i in range(20):
    total = total + lancer_de()
print("moyenne obtenue :", total / 20)
```

La moyenne affichée change à chaque exécution, mais elle se rapproche de 3,5 lorsqu'on augmente le nombre de lancers. Nous verrons au chapitre 8 comment cette observation illustre la loi des grands nombres.

Chapitre 7

Les listes

Montrez-moi vos organigrammes en me cachant vos tables, je resterai perplexe. Montrez-moi vos tables, je n'aurai en général pas besoin de vos organigrammes.

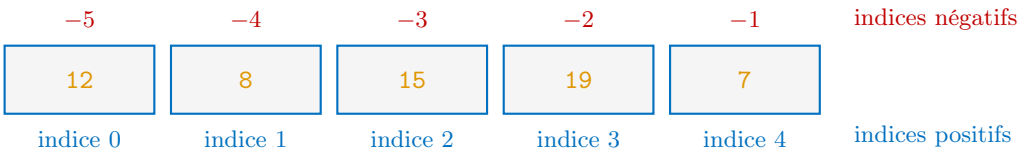
Frederick P. Brooks - *The Mythical Man-Month*, 1975, chapitre 9, p. 102

Une variable ne retient qu'une valeur à la fois. Au chapitre 4, le calcul des termes d'une suite écrasait chaque terme par le suivant : à la fin, tout avait disparu sauf le dernier. Or nous avons souvent besoin de conserver l'ensemble des valeurs, pour les comparer, les trier ou les représenter graphiquement. C'est le rôle de la liste, première *structure de données* de ce cours.

7.1 Créer et lire une liste

> **Définition :** Une **liste** est une suite ordonnée de valeurs, écrite entre crochets et séparée par des virgules. Chaque valeur occupe une position repérée par un **indice**, et l'on accède à la valeur d'indice i de la liste `L` par l'écriture `L[i]`.

```
notes = [12, 8, 15, 19, 7]
print(notes[0])     # 12, le premier élément
print(notes[4])     # 7, le dernier
print(len(notes))   # 5, le nombre d'éléments
```



La figure met en évidence le point qui déroute le plus : la numérotation commence à **zéro**. Une liste de n éléments a donc pour indices $0, 1, \ldots, n-1$, exactement les entiers de `range(n)` du chapitre 4, ce qui n'est pas une coïncidence. Les indices négatifs, eux, comptent depuis la fin : `L[-1]` désigne toujours le dernier élément, quelle que soit la longueur de la liste.

Remarque : L'erreur la plus répandue est l'`IndexError: list index out of range`, obtenue en demandant `notes[5]` sur une liste de cinq éléments. Le dernier indice valide est 4. Le réflexe à acquérir est d'écrire `L[len(L)-1]`, ou plus simplement `L[-1]`, pour atteindre le dernier terme.

Remarque : Ce décalage entre l'indice informatique et le rang mathématique demande de la vigilance. Si l'on range les termes $u_0, u_1, \ldots$ d'une suite dans une liste, tout va bien : `U[n]` contient u_n. Mais si l'on range les termes $v_1, v_2, \ldots$ d'une suite commençant au rang 1, alors `V[0]` contient v_1, et tous les indices sont décalés d'une unité. Décidez de la convention avant d'écrire le programme, et notez-la en commentaire.

7.2 Modifier une liste

Propriété : Contrairement à un nombre ou à une chaîne, une liste est **modifiable** : on peut changer la valeur d'une case sans reconstruire la liste entière.

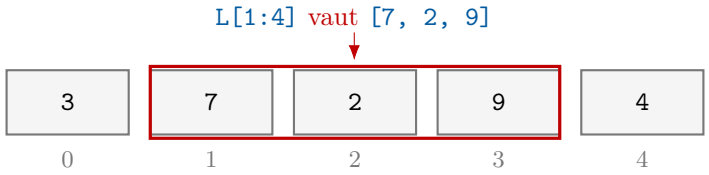
```
notes = [12, 8, 15, 19, 7]
notes[1] = 11          # la deuxième note est corrigée
notes.append(16)       # une sixième note est ajoutée à la fin
print(notes)           # [12, 11, 15, 19, 7, 16]
```

L'instruction `append` est la plus utile de toutes : elle allonge la liste d'un élément. Elle s'écrit *après* le nom de la liste, séparée par un point, car c'est une **méthode**, c'est-à-dire une fonction attachée à l'objet sur lequel elle agit.

Remarque : Une méthode comme `append` modifie la liste sur place et ne renvoie rien. Écrire `notes = notes.append(16)` est donc une faute classique : la variable `notes` reçoit `None` et la liste est perdue. On écrit simplement `notes.append(16)`, sans affectation.

7.3 Extraire une tranche

Définition : L'écriture `L[a:b]` désigne la **tranche** formée des éléments d'indices a à $b-1$. Elle constitue une *nouvelle* liste ; l'originale n'est pas modifiée.



La borne de gauche est incluse, celle de droite exclue : c'est la convention déjà rencontrée avec `range`, et la tranche compte donc $b-a$ éléments. On peut omettre une borne : `L[:3]` part du début et vaut `[3, 7, 2]`, tandis que `L[2:]` va jusqu'à la fin et vaut `[2, 9, 4]`.

Remarque : L'écriture `L[:]` désigne la liste entière, et sert à en fabriquer une copie indépendante. Nous verrons à la fin du chapitre pourquoi cette précaution est parfois vitale.

7.4 Construire une liste avec une boucle

♡ **Méthode :** Pour construire une liste, on part de la liste vide et on ajoute les éléments un à un : c'est le schéma d'accumulation du chapitre 4, appliqué à une liste.

```
carres = []
for k in range(1, 8):
    carres.append(k * k)
print(carres)          # [1, 4, 9, 16, 25, 36, 49]
```

Exemple : Reprenons la suite du chapitre 4, définie par $u_0 = 2$ et $u_{n+1} = 0{,}8\,u_n + 3$, mais en conservant cette fois tous les termes.

```
U = [2]
for n in range(5):
    U.append(0.8 * U[n] + 3)
print(U)
```

La liste obtenue est `[2, 4.6, 6.68, 8.344000000000001, 9.6752, 10.74016]`. L'écriture `U[n]` donne directement le terme u_n : la traduction de la relation de récurrence est immédiate, et l'on dispose à la fin de tous les termes, par exemple pour les représenter graphiquement.

Remarque : Le troisième terme s'affiche `8.344000000000001` et non `8.344` : c'est l'approximation binaire des `float` signalée au chapitre 2 qui refait surface, l'affichage d'une liste ne procédant à aucun arrondi de confort. Pour une présentation lisible, on construit une liste arrondie avec `[round(u, 3) for u in U]`, mais on conserve la liste exacte pour les calculs : arrondir en cours de route dégraderait tous les termes suivants.

Remarque : Python offre une écriture condensée, appelée **liste en compréhension**, qui rappelle la notation $\{k^2 \mid 1 \leqslant k \leqslant 7\}$ des mathématiques :

```
carres = [k * k for k in range(1, 8)]
```

Elle est élégante et fréquente, mais réservez-la aux cas simples : dès qu'un test ou deux instructions interviennent, la boucle explicite reste plus lisible.

7.5 Parcourir une liste

Il y a deux manières de parcourir une liste, et le choix entre les deux n'est pas une affaire de goût.

```
temperatures = [12, 15, 9, 21, 18]

for t in temperatures:          # parcours par élément
    print(t)

for i in range(len(temperatures)):    # parcours par indice
    print("jour", i + 1, ":", temperatures[i])
```

♡ **Méthode :** Parcourez **par élément** lorsque seules les valeurs comptent : c'est plus court et plus sûr. Parcourez **par indice** lorsque vous avez besoin de la position, pour l'afficher, pour modifier la liste, ou pour comparer un élément à son voisin.

Remarque : Dans le parcours par élément, la variable `t` est une copie de la valeur : lui affecter `t = 0` ne change rien à la liste. Pour modifier réellement le contenu, il faut passer par l'indice et écrire `temperatures[i] = 0`.

Instruction	Effet	Exemple sur `[3, 7, 2, 9, 4]`
`len(L)`	nombre d'éléments	5
`sum(L)`	somme des éléments	25
`min(L)`, `max(L)`	plus petit, plus grand	2 et 9
`sorted(L)`	nouvelle liste triée	`[2, 3, 4, 7, 9]`
`9 in L`	appartenance (booléen)	`True`
`L + M`	concaténation	met les deux listes bout à bout

Remarque : Ces fonctions dispensent d'écrire des boucles, mais ne dispensent pas de savoir les écrire : au chapitre 8, nous programmerons nous-mêmes la recherche du maximum, car c'est l'algorithme, et non l'instruction, qui fait l'objet du programme de mathématiques.

7.6 Un piège : deux noms pour une même liste

```
A = [1, 2, 3]
B = A
B.append(4)
print(A)      # [1, 2, 3, 4] et non [1, 2, 3] !
```



L'affectation `B = A` ne recopie pas la liste : elle donne un second nom à la même zone de mémoire, si bien que toute modification faite par l'un est visible par l'autre. C'est la nuance annoncée au chapitre 2 : pour les nombres, l'affectation copie la valeur ; pour les listes, elle copie seulement l'adresse.

♡ **Méthode :** Pour obtenir une copie véritablement indépendante, utilisez la tranche complète ou la fonction `list` :

```
B = A[:]          # ou bien : B = list(A)
B.append(4)
print(A)          # [1, 2, 3] : A est intacte
```

Remarque : Ce piège explique une classe entière de bugs difficiles à trouver : une fonction reçoit une liste en paramètre, la modifie, et le programme principal se retrouve avec des données altérées sans qu'aucune affectation n'apparaisse. Si une fonction doit transformer une liste, faites-lui-en prendre une copie au début, ou renvoyez une nouvelle liste plutôt que de modifier l'ancienne.

Chapitre 8

Quelques algorithmes classiques

Le mot *algorithme* vient du nom du savant persan **Muhammad ibn Mūsā al-Khwārizmī**, actif à Bagdad vers 820, dont les traités d'arithmétique et d'algèbre — c'est à lui que l'on doit également le mot *algèbre* — exposaient des procédés de calcul systématiques. Un algorithme est en effet une méthode : une suite finie d'opérations non ambiguës conduisant au résultat en un nombre fini d'étapes. Ce dernier chapitre rassemble six algorithmes classiques qui mobilisent tout ce qui précède, et dont plusieurs figurent explicitement au programme du lycée.

8.1 Le maximum d'une liste

♡ **Méthode :** On parcourt la liste en conservant dans une variable le plus grand élément rencontré jusque-là. Cette variable est initialisée avec le premier élément, jamais avec zéro.

```
def maximum(L):
    """Renvoie le plus grand élément de la liste non vide L."""
    m = L[0]
    for x in L:
        if x > m:
            m = x
    return m

print(maximum([3, 7, 2, 9, 4]))     # 9
```

Remarque : Initialiser `m` à 0 serait une faute : sur la liste `[-5, -3, -8]`, l'algorithme renverrait 0, qui n'est même pas un élément de la liste. C'est l'exemple type de l'erreur qui ne se voit pas tant qu'on ne teste qu'avec des nombres positifs.

Remarque : Pour obtenir non pas le maximum mais sa *position*, on mémorise l'indice plutôt que la valeur, ce qui impose le parcours par indice du chapitre 7 : on remplace la boucle par `for i in range(len(L))` et le test par `if L[i] > L[imax]`.

8.2 Balayage : encadrer une solution

Soit f la fonction définie sur $[0\,;1]$ par $f(x) = x^3 + x - 1$. Elle est continue et strictement croissante, avec $f(0) = -1 < 0$ et $f(1) = 1 > 0$: d'après le théorème des valeurs intermédiaires, l'équation $f(x) = 0$ admet une unique solution α dans $[0\,;1]$. Aucune formule simple ne la donne ; nous allons l'approcher.

```
def f(x):
    return x**3 + x - 1

pas = 0.01
k = 0
while f(k * pas) <= 0:
    k = k + 1
print("alpha est compris entre", (k-1) * pas, "et", k * pas)
```

Le programme affiche que α est compris entre 0,68 et 0,69. Le principe est celui du balayage : on avance pas à pas jusqu'à ce que le signe change, et l'encadrement obtenu a pour amplitude le pas choisi.

Remarque : On remarquera que le compteur `k` est un entier et que l'abscisse est reconstruite par `k * pas`. Écrire `x = x + 0.01` à la place cumulerait les erreurs d'arrondi signalées au chapitre 2, au point qu'après cent additions la variable ne vaudrait pas exactement 1. Compter en entiers et convertir au dernier moment est une règle générale de prudence.

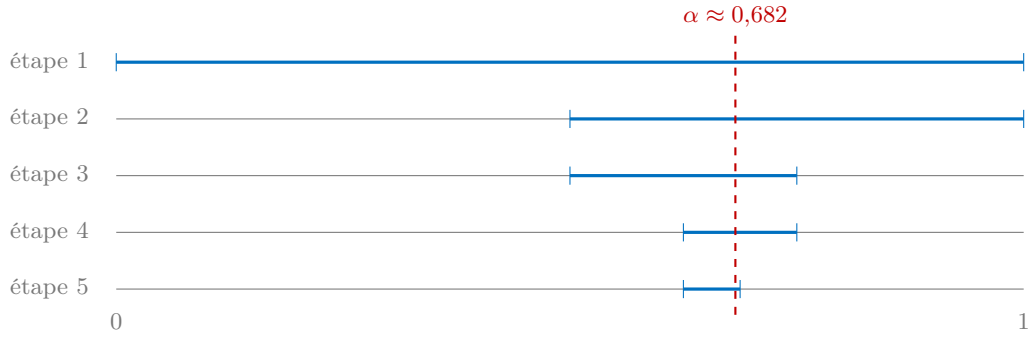
Remarque : Pour un encadrement d'amplitude 10^{-6}, le balayage demanderait environ un million de tours. C'est encore faisable, mais l'algorithme suivant obtient le même résultat en vingt étapes.

8.3 La dichotomie

♡ **Méthode :** On part d'un intervalle $[a\,;b]$ contenant la solution, on calcule son milieu m, et l'on conserve celle des deux moitiés qui contient encore la solution, repérée par le changement de signe. La longueur de l'intervalle est divisée par deux à chaque étape.

```
a, b = 0, 1
while b - a > 0.001:
    m = (a + b) / 2
    if f(m) <= 0:
        a = m
    else:
        b = m
print(a, b)
```

Étape	a	b	m	$f(m)$	On garde
1	0	1	0,5	−0,375	la moitié droite
2	0,5	1	0,75	0,1719	la moitié gauche
3	0,5	0,75	0,625	−0,1309	la moitié droite
4	0,625	0,75	0,6875	0,0125	la moitié gauche
5	0,625	0,6875	0,65625	−0,0611	la moitié droite



La figure montre les intervalles successifs, tous emboîtés et tous contenant la droite verticale d'abscisse α : c'est la garantie de la méthode. Après n étapes, l'amplitude vaut $\frac{1}{2^n}$, si bien que dix étapes donnent déjà trois décimales exactes et vingt étapes environ six.

Remarque : La condition d'arrêt porte sur l'amplitude `b - a`, et non sur la valeur de `f(m)`. Tester `f(m) == 0` serait vain : la solution est irrationnelle, et aucun `float` ne l'atteindra jamais exactement. C'est une application directe de la mise en garde du chapitre 3 sur la comparaison des nombres à virgule.

8.4 Le tri par sélection

Trier une liste, c'est en réorganiser les éléments dans l'ordre croissant. La méthode la plus naturelle consiste à chercher le plus petit élément et à le mettre en première position, puis à recommencer sur ce qui reste.

```
def tri_selection(L):
    for i in range(len(L)):
        j_min = i
        for j in range(i + 1, len(L)):
            if L[j] < L[j_min]:
                j_min = j
        L[i], L[j_min] = L[j_min], L[i]
    return L

print(tri_selection([5, 3, 8, 1, 9, 2]))
```

Étape	Liste après échange	Commentaire
départ	`[5, 3, 8, 1, 9, 2]`	—
$i = 0$	`[1, 3, 8, 5, 9, 2]`	le minimum 1 prend la place de 5
$i = 1$	`[1, 2, 8, 5, 9, 3]`	le minimum du reste est 2
$i = 2$	`[1, 2, 3, 5, 9, 8]`	puis 3
$i = 3$	`[1, 2, 3, 5, 9, 8]`	5 est déjà bien placé
$i = 4$	`[1, 2, 3, 5, 8, 9]`	dernier échange

On lit dans ce tableau l'invariant de l'algorithme : après l'étape i, les $i + 1$ premières cases contiennent, dans l'ordre, les $i + 1$ plus petits éléments. C'est cette propriété, vraie à chaque tour, qui démontre que l'algorithme trie effectivement la liste.

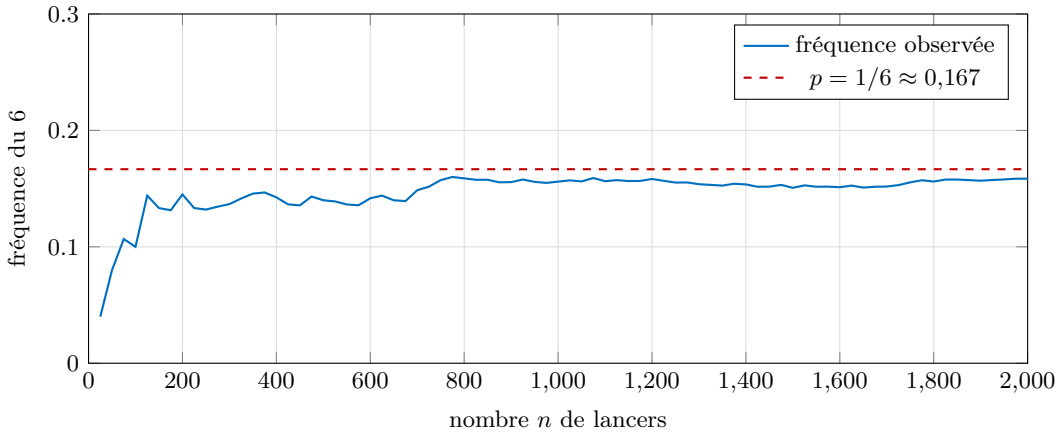
Remarque : Les deux boucles imbriquées font que le nombre de comparaisons est de l'ordre de $\frac{n^2}{2}$ pour une liste de n éléments : trier mille valeurs demande un demi-million de comparaisons. Les tris employés en pratique, comme celui de la fonction `sorted` du chapitre 7, sont bien plus rapides, mais leur principe dépasse le cadre de ce cours.

Remarque : Cette fonction modifie la liste reçue en paramètre, conformément au piège signalé au chapitre 7 : après l'appel, la liste d'origine est triée elle aussi. Pour préserver l'original, commencez la fonction par `L = L[:]`.

8.5 Simulation et loi des grands nombres

```
import random

def frequence_de_six(n):
    """Fréquence d'apparition du 6 sur n lancers d'un dé équilibré."""
    compteur = 0
    for i in range(n):
        if random.randint(1, 6) == 6:
            compteur = compteur + 1
    return compteur / n
```



Cette courbe est le résultat d'une simulation réelle de 2000 lancers. Elle illustre la loi des grands nombres : la fréquence observée est très instable au début — après 25 lancers elle vaut 0,04 — puis se stabilise autour de la probabilité théorique $\frac{1}{6}$. Elle ne l'atteint pas exactement et ne s'y « colle » jamais définitivement : la loi affirme seulement que l'écart devient improbable, non qu'il devient nul.

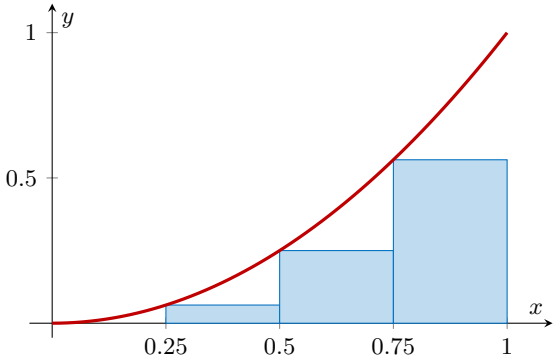
Remarque : Deux exécutions successives donnent deux courbes différentes, puisque les tirages sont aléatoires. C'est une propriété du programme, non un défaut. Pour retrouver exactement la même suite de tirages, par exemple afin de comparer deux versions d'un algorithme, on fixe la graine du générateur par `random.seed(7)` au début du programme.

8.6 L'aire sous une courbe

Terminons par un algorithme qui relie la programmation au calcul intégral : l'approximation de $\int_0^1 x^2\,\mathrm{d}x$ par la méthode des rectangles.

```
def rectangles(n):
    """Somme des aires des n rectangles à gauche sur [0 ; 1]."""
    aire = 0
    for k in range(n):
        aire = aire + (k / n)**2 * (1 / n)
    return aire

for n in [4, 10, 100, 1000]:
    print(n, rectangles(n))
```



n	4	10	100	1000
valeur approchée	0,21875	0,285	0,32835	0,3328335

Les valeurs se rapprochent de $\frac{1}{3} \approx 0{,}3333$, qui est la valeur exacte de l'intégrale. Elles restent toutes inférieures à cette limite, ce qui n'a rien d'accidentel : la fonction carré étant croissante, chaque rectangle appuyé sur la borne de gauche passe sous la courbe, comme le montre la figure. C'est exactement la construction qui sert à *définir* l'intégrale.

Remarque : L'erreur commise est de l'ordre de $\frac{1}{n}$: il faut multiplier le nombre de rectangles par dix pour gagner une décimale. Ce n'est pas très efficace, et il existe de meilleures méthodes, mais c'est la plus simple à programmer et la seule qui traduise littéralement la définition.

Annexe A

Erreurs fréquentes et mémento

A.1 Les messages d'erreur les plus courants

Le tableau suivant rassemble les erreurs rencontrées tout au long du cours. La colonne de droite indique le premier réflexe à avoir, avant toute autre investigation.

Message	Cause habituelle	Que faire
`SyntaxError`	phrase mal formée : deux-points ou parenthèse oubliés	relire la ligne indiquée *et celle qui la précède*
`IndentationError`	décalage incohérent des lignes d'un bloc	aligner exactement le bloc, sans mélanger espaces et tabulations
`NameError`	nom inconnu : faute de frappe, ou variable employée avant d'être définie	vérifier l'orthographe et l'ordre des lignes
`TypeError`	opération entre types incompatibles, comme `"2" + 3`	convertir avec `int`, `float` ou `str`
`ValueError`	conversion impossible, comme `int("douze")`	contrôler ce que l'utilisateur a saisi
`IndexError`	indice hors de la liste	se souvenir que le dernier indice est `len(L)-1`
`ZeroDivisionError`	division par zéro	tester le dénominateur avant de diviser
`AttributeError`	méthode inexistante pour ce type, comme `append` sur un nombre	vérifier le type de l'objet avec `type()`

Remarque : Aucun de ces messages n'apparaît lorsque le programme est simplement *faux*. Un résultat absurde, une boucle qui tourne indéfiniment, une valeur décalée d'une unité : ce sont les erreurs les plus coûteuses, et le seul remède est le tableau de trace du chapitre 4, complété par des `print` temporaires placés à l'intérieur des boucles.

A.2 Mémento

Bases (chapitres 1 et 2)	
`print(a, b)`	affiche les valeurs séparées par une espace
`x = 5`	range 5 dans la variable `x`
`a, b = b, a`	échange les contenus de `a` et `b`
`int(v)`, `float(v)`, `str(v)`	conversions de type
`7 // 2`, `7 % 2`	quotient et reste de la division euclidienne
`import math`	donne accès à `math.sqrt`, `math.pi`, `math.floor`
Tests (chapitre 3)	
`==`, `!=`, `<=`, `>=`	comparaisons
`and`, `or`, `not`	connecteurs logiques
`if ... : / elif ... : / else :`	disjonction de cas
Boucles (chapitres 4 et 5)	
`for i in range(a, b):`	répète pour $i = a$ jusqu'à $b - 1$
`while condition :`	répète tant que la condition est vraie
Fonctions (chapitre 6)	
`def nom(p1, p2):`	définit une fonction et ses paramètres
`return v`	renvoie la valeur `v` et termine la fonction
`random.randint(1, 6)`	entier au hasard, **les deux bornes incluses**
Listes (chapitre 7)	
`L = [3, 7, 2]`	crée une liste ; premier indice 0
`L[i]`, `L[-1]`, `L[a:b]`	élément, dernier élément, tranche
`L.append(v)`	ajoute `v` à la fin
`len(L)`, `sum(L)`, `sorted(L)`	longueur, somme, copie triée
`M = L[:]`	copie indépendante de `L`

A.3 Six règles pour programmer sans souffrir

1. **Écrire l'algorithme en français avant de le taper.** Trois phrases suffisent le plus souvent, et elles évitent d'écrire vingt lignes dans la mauvaise direction.
2. **Tester sur un cas dont on connaît la réponse.** Un programme qui calcule une somme doit d'abord être essayé sur la somme des dix premiers entiers, dont on sait qu'elle vaut 55.
3. **Avancer par petits morceaux.** Écrire cinq lignes, les exécuter, puis continuer : une erreur dans cinq lignes se trouve en une minute, la même erreur dans cinquante lignes en une heure.
4. **Lire le message d'erreur en entier**, en commençant par la dernière ligne, comme au chapitre 1.
5. **Faire un tableau de trace** dès que le résultat est inattendu, plutôt que de modifier le programme au hasard en espérant que cela passe.
6. **Nommer les variables pour un lecteur humain**, ainsi que le recommandait Martin Fowler en tête du chapitre 6 : le premier lecteur de votre programme, ce sera vous, dans deux mois.

Annexe B

Deux environnements pour programmer

Écrire un programme suppose un endroit où l'écrire et un interpréteur pour l'exécuter. Deux solutions s'offrent à vous, et elles ne s'excluent pas : un éditeur en ligne, immédiatement disponible dans un navigateur, et un logiciel installé sur votre machine. La première ne demande aucune installation mais ne conserve rien ; la seconde demande cinq minutes de mise en place mais devient votre outil de travail pour l'année. Cette annexe décrit l'une puis l'autre.

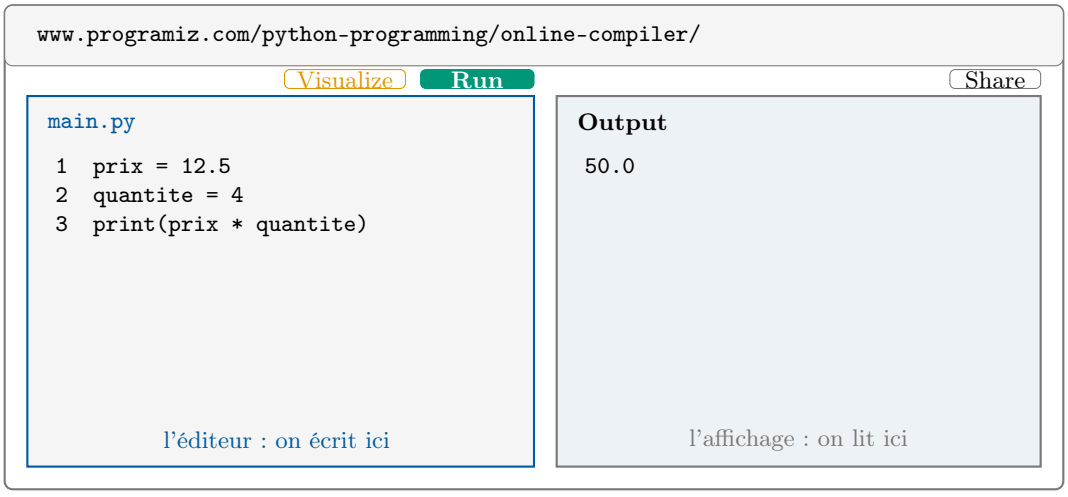
B.1 Programiz, dans le navigateur

B.1.1 Ouvrir l'éditeur

Il n'y a rien à installer ni de compte à créer : il suffit d'ouvrir l'adresse

`https://www.programiz.com/python-programming/online-compiler/`

dans n'importe quel navigateur. La page se présente en deux volets, et c'est toute sa force : à gauche l'éditeur, où l'on écrit le programme, à droite la zone d'affichage, où apparaît ce que le programme produit.



Cette figure schématise ce que vous verrez : le nom de fichier `main.py` en haut du volet gauche, le bouton `Run` qui lance l'exécution, et le mot `Output` en tête du volet droit. Le programme donné en exemple est celui du chapitre 2 ; il affiche `50.0`.

♡ **Méthode :** Pour exécuter un premier programme : effacez le code d'exemple présent à l'ouverture, tapez le vôtre dans le volet de gauche, cliquez sur `Run`, et lisez le résultat dans le volet de droite. Si le programme contient une erreur, c'est également dans ce volet que s'affiche le message étudié au chapitre 1 et répertorié à l'annexe A.

Remarque : Les programmes qui font appel à `input`, vus au chapitre 2, fonctionnent parfaitement : la question s'affiche dans le volet de droite, et c'est là que vous tapez la réponse avant de valider par Entrée.

B.1.2 Suivre l'exécution pas à pas

Le bouton `Visualize` ouvre un mode particulièrement utile en début d'apprentissage : le programme n'est plus exécuté d'un coup, mais ligne par ligne, avec l'affichage de la valeur de chaque variable à chaque étape. C'est, sous forme animée, exactement le tableau de trace du chapitre 4.

Remarque : Prenez l'habitude d'y recourir devant un résultat inattendu, plutôt que de modifier le programme au hasard. Voir de ses yeux l'accumulateur repartir de zéro à chaque tour vaut toutes les explications sur l'initialisation.

B.1.3 Les limites à connaître

Remarque : **Rien n'est conservé.** L'outil n'enregistre pas votre travail : fermer l'onglet, ou parfois simplement recharger la page, efface tout. Avant de quitter, sélectionnez votre code, copiez-le et collez-le dans un fichier texte, ou utilisez le bouton `Share`, qui fabrique un lien vers votre programme. C'est la raison pour laquelle cet outil convient à un essai de dix minutes, non à un devoir que l'on reprend le lendemain.

Remarque : **Un seul fichier, et pas d'accès au disque.** Tout tient dans `main.py` : on ne peut pas répartir un projet en plusieurs fichiers, ni lire un fichier de données situé sur votre ordinateur. Pour tout ce que nous faisons dans ce cours, c'est sans conséquence.

Remarque : **Le code est exécuté sur un serveur distant.** Une connexion est donc nécessaire, et un programme trop long finit par être interrompu. Une boucle infinie, au sens du chapitre 5, s'y arrête d'elle-même au bout de quelques secondes — ce qui, soit dit en passant, est une façon confortable de découvrir l'erreur sans figer sa machine.

B.2 Thonny, sur son ordinateur

Thonny est un environnement de développement conçu pour l'enseignement, gratuit et libre, écrit à l'université de Tartu par l'Estonien Aivar Annamaa et diffusé depuis 2014. Il présente pour un débutant un avantage décisif sur ses concurrents : il est livré avec Python, si bien qu'une seule installation suffit, et son interface ne montre que l'essentiel.

B.2.1 Installer le logiciel

♡ **Méthode :** L'installation se fait en trois temps, identiques sur les trois systèmes.

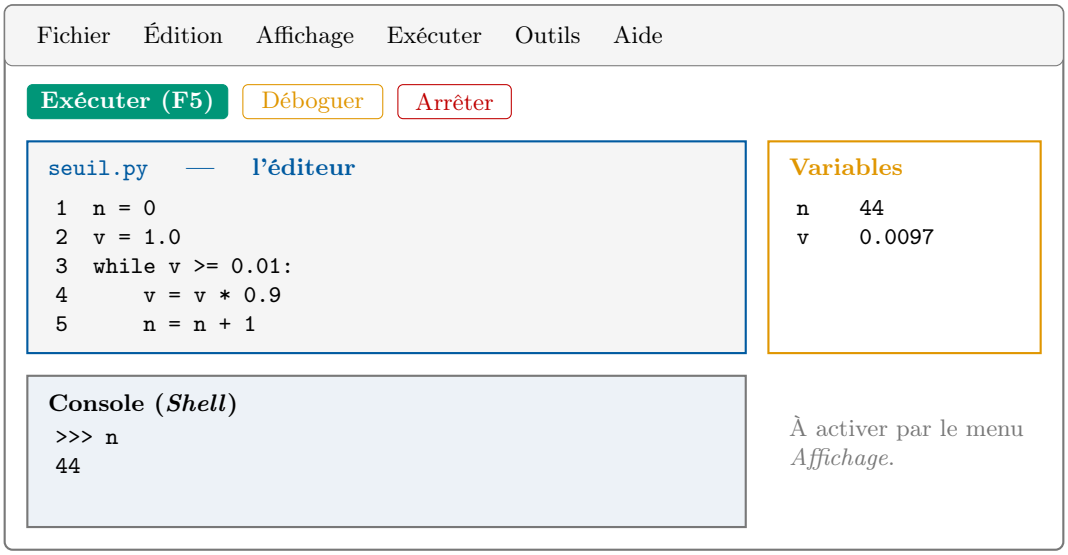
1. Ouvrir l'adresse `https://thonny.org` et cliquer sur le lien correspondant à votre système, en haut à droite de la page.
2. Lancer le fichier téléchargé : un `.exe` sous Windows, un `.pkg` sous macOS. Sous Linux, une seule ligne suffit dans un terminal : `sudo apt install thonny`.
3. Accepter les étapes proposées sans rien modifier, puis ouvrir Thonny depuis le menu Démarrer, le dossier Applications ou le terminal.

Au premier démarrage, Thonny demande la langue de l'interface : le français est disponible. Il propose également un mode d'installation « pour tous les utilisateurs » ou « pour moi seulement » ; sur un ordinateur personnel, le second évite d'avoir à saisir un mot de passe d'administrateur.

Remarque : Au moment où ces lignes sont écrites, la version diffusée est la 5.0.0, publiée en avril 2026, qui embarque Python 3.14 et demande Windows 10 ou 11. Sur une machine plus ancienne, la version 4 reste téléchargeable depuis la même page et convient parfaitement à tout ce qui figure dans ce cours. Vérifiez le numéro dans le menu *Aide* puis *À propos*.

Remarque : Aucune installation séparée de Python n'est nécessaire, et il est même déconseillé d'en ajouter une : deux Python sur la même machine sont la première cause de « mais ça marchait hier » chez les débutants.

B.2.2 L'interface



La fenêtre se lit en trois zones : l'éditeur en haut, où l'on écrit et enregistre le programme ; la console en bas, qui joue le rôle du mode interactif décrit au chapitre 1 et affiche les résultats et les messages d'erreur ; et, si on l'active, le panneau des variables sur la droite.

B.2.3 Écrire et exécuter un premier programme

♡ **Méthode :** Tapez votre programme dans l'éditeur, puis enregistrez-le par `Ctrl + S` en lui donnant un nom suivi de l'extension `.py`. Appuyez enfin sur la touche `F5`, ou cliquez sur le bouton d'exécution : le programme s'exécute, et tout ce qu'il affiche apparaît dans la console.

Remarque : Thonny refuse d'exécuter un programme non enregistré et propose de le faire pour vous. Ce n'est pas une brimade : l'exécution porte sur le fichier, et non sur ce qui est visible à l'écran. Un programme modifié mais non enregistré produirait l'ancien résultat, ce qui est la plus déroutante des situations.

Remarque : L'erreur la plus répandue est de baptiser son fichier du nom d'un module. Un fichier nommé `math.py` ou `random.py` sera chargé à la place du véritable module lors de l'instruction `import`, et le programme échouera sur un message incompréhensible du genre « le module n'a pas d'attribut `sqrt` ». Nommez vos fichiers d'après ce qu'ils font : `suite_seuil.py`, `tri.py`.

Remarque : Si vous avez écrit une boucle infinie, au sens du chapitre 5, la console reste bloquée. Le bouton rouge d'arrêt de la barre d'outils, ou la combinaison `Ctrl + C` dans la console, interrompt le programme et vous rend la main.

B.2.4 Le débogueur, ou le tableau de trace automatique

C'est la fonction qui justifie à elle seule le choix de Thonny pour un cours de mathématiques. Le bouton *Déboguer*, puis les touches de progression pas à pas, exécutent le programme *une ligne à la fois* : la ligne sur le point d'être exécutée est surlignée, et le panneau des variables montre leur contenu se modifier.

♡ **Méthode :** Devant un programme qui ne donne pas le résultat attendu :

1. ouvrir le panneau *Variables* par le menu *Affichage* ;
2. lancer l'exécution en mode *Déboguer* au lieu du mode normal ;
3. avancer pas à pas et comparer, à chaque tour de boucle, les valeurs affichées avec celles du tableau de trace que vous avez établi à la main.

La première ligne où les deux divergent est la ligne fautive.

Remarque : Thonny comporte aussi un *Assistant*, panneau qui commente les messages d'erreur en langue courante et suggère des causes probables. C'est une aide précieuse, à condition de lire d'abord le message lui-même : l'annexe A vous rendra plus autonome que n'importe quel assistant.

B.3 Lequel choisir ?

	Programiz, en ligne	**Thonny, en local**
Installation	aucune	quelques minutes, une seule fois
Enregistrement	aucun : à recopier soi-même	fichiers `.py` sur votre disque
Connexion	indispensable	inutile
Exécution pas à pas	oui, bouton `Visualize`	oui, avec le panneau des variables
Bon pour	un essai rapide, une machine qui n'est pas la vôtre	le travail de l'année, les devoirs

La réponse raisonnable est donc : les deux. Installez Thonny chez vous et travaillez-y ; gardez Programiz pour vérifier une idée en trois lignes depuis n'importe quel poste. Dans les deux cas, le Python est le même, et tous les programmes de ce cours s'y exécutent à l'identique.